Gayatri AI → Mock Exam Portal

Conversion Plan & 15-Day Development Documentation

Source repo: `github.com/Gayatri-Education/Gayatri-AI` (public, Flet + LM Studio, v5.0) **Target:** Multi-user web-based mock test platform for competitive exams (SSC, Banking, Railways, UPSC-type prelims, etc.) **Team:** 8 people — 2 AI/ML Developers, 2 Data Analysts, 2 Full-Stack Developers, 2 RPA/Python Developers **Duration:** 15 working days

---

Part 1 — What the Current Repo Actually Does

The repo is a **single-user Windows desktop chat app**, built with Flet (Flutter-for-Python), that talks to a locally running LM Studio server. It has no concept of multiple users, timed sessions, question banks, or scoring — it is a chat assistant, not a testing engine. Understanding this gap is the whole point of the plan below: we are not "adding a feature," we are **repurposing the backend plumbing** and **discarding the desktop chat UI entirely**.

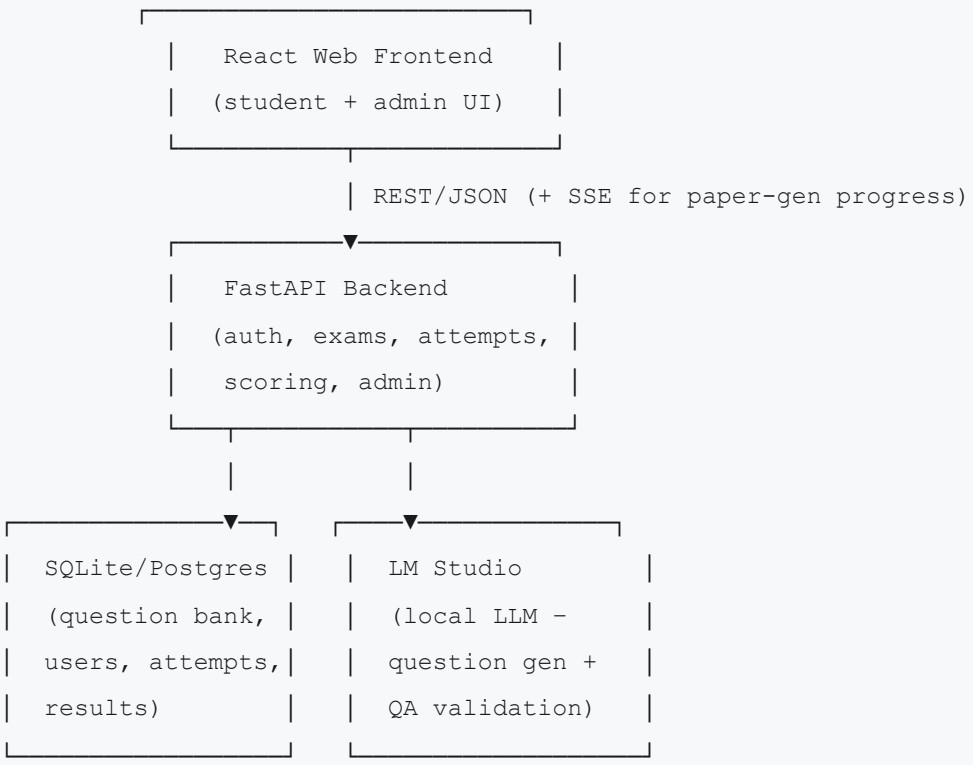
Module-by-module audit

File	What it does today	Reusable for exam portal?
`main.py`	Flet app entrypoint — builds the desktop window, wires UI components together	❌ No — desktop-only, single window, single user. Replaced entirely.
`ui_components.py`	All Flet chat UI widgets (sidebar, message bubbles, theme toggle)	❌ No — wrong UI paradigm (chat, not exam). Replaced by a React web frontend.
`sample_ui.py`, `test.py`	Scratch/demo files	❌ No — dead weight, delete.
`config.py`	App-wide constants (LM Studio URL, model name, token budgets, sidebar width)	✅ Partial — the *pattern* (central config module) is kept; exam-specific settings replace UI settings.
`llm_client.py`	Streaming client that talks to LM Studio's OpenAI-compatible `/v1/chat/completions` endpoint	✅ Yes, heavily reused — this becomes the engine that generates and validates exam questions. Needs a new "structured JSON output" mode added.
`search_engine.py`	Web search + URL crawling to ground chat answers	⚠️ Optional/deferred — could ground GK/current-affairs question generation later, but not needed for MVP. Kept but not wired in for the first 15 days.
`context_manager.py`	RAG: chunks + embeds Markdown files, ranks relevant chunks for a query	✅ Yes, repurposed — instead of "chat context files," it will feed syllabus/reference documents to the question generator so generated questions stay on-topic.
`db.py`	SQLite layer for chat sessions/messages (single local user)	❌ Schema not reusable (chat-shaped, not exam-shaped) but the *module pattern* (a thin DB access layer) is kept. Full rewrite of the schema (Part 3).
`diagnostic_test.py`	Connectivity/system diagnostics for the desktop app	⚠️ Adapted into a `/health` API check for the new backend, otherwise not reused.
`gayatri.db`	Committed SQLite file with local chat history	❌ Remove from git tracking — a testing platform's DB must never ship pre-seeded personal data or live in version control.
`requirements.txt`	Flet + LM Studio client deps	✅ Replaced with a FastAPI/React dependency set (Part 2).

Bottom line: ~2 of 10 files survive close to as-is (`llm_client.py`, `context_manager.py`). Everything UI- and DB-facing is new. This is effectively a rewrite that reuses the AI-calling layer, not an incremental feature addition — the team should be scoped accordingly, which is why the plan below front-loads architecture agreement on Day 1–2.

Part 2 — Target Architecture

Why a rewrite of the UI layer is mandatory: the flow described — *"user comes, selects exam, we locally prepare a question paper, user takes exam, we publish results"* — requires multiple concurrent users, server-side timers, tamper-resistant scoring, and an admin/analytics view. A single-window desktop Flet app cannot serve multiple simultaneous test-takers. This has to become a client-server web app.



Stack decisions

- **Backend:** FastAPI (Python) — keeps the team's existing Python skillset, and `llm_client.py`'s async streaming pattern ports over almost directly.
- **Frontend:** React + Vite + TailwindCSS — same stack already standardized on for the Gayatri AI product family, so the full-stack devs aren't learning a second frontend paradigm.
- **DB:** SQLite for the 15-day MVP (zero-ops, matches existing `db.py` pattern), with the schema written in SQLAlchemy so a Postgres swap later is a config change, not a rewrite. **Flag this explicitly to the client/stakeholder as an MVP constraint** — SQLite's single-writer lock will need replacing before large concurrent live-exam cohorts (see Day 14 note).
- **Question source: hybrid** — a curated, human/RPA-built question bank (reliable, matches real exam patterns) topped up by LLM-generated questions where bank coverage is thin (flexible, scales to new exam types fast). Pure on-the-fly LLM generation for every question is explicitly rejected for MVP: it's slower, harder to guarantee correctness on, and competitive-exam takers will punish wrong answer keys immediately.
- **Auth:** simple email/phone + password, JWT session tokens. No SSO/OTP in MVP scope.

- **Timer/integrity:** exam duration enforced **server-side** (attempt has a `started_at` + `duration_seconds`; the API rejects/auto-submits after expiry) — never trust a client-side JS timer alone for scoring cutoffs.

Part 3 — Database Schema (new)

Replaces `db.py`'s chat schema entirely.

Table	Key columns	Purpose
`users`	id, name, email, password_hash, role (`student` / `admin`)	Auth
`exams`	id, name, category (SSC/Banking/Railways/...), description	The catalog of selectable exams
`exam_blueprints`	id, exam*id, section*name, num*questions, marks*per*q, *negative*marks, duration_minutes	Defines a paper's structure per exam — owned by Data Analysts
`topics`	id, exam_id, name	Topic/section tagging for question bank + weak-area reports
`questions`	id, exam*id, topic*id, text, option*a–d, correct*option, difficulty, source (`bank` / `llm`), explanation	The question bank
`papers`	id, exam*id, generated*at, seed	One generated paper instance
`paper_questions`	paper*id, question*id, order_index	Which questions are on which paper, in what order
`attempts`	id, user*id, paper*id, started*at, duration*seconds, submitted_at, status (`in_progress` / `submitted` / `auto_submitted`)	One student's exam session
`attempt_answers`	attempt*id, question*id, selected*option, is*correct, time*spent*seconds	Per-question response log
`results`	attempt*id, raw*score, max*score, percentile, topic*breakdown_json	Published, read-optimized result record

---

Part 4 — Question Paper Preparation Pipeline

1. **Blueprint lookup** — for the exam the user selected, read `exam_blueprints` (e.g., "SSC CGL Tier 1: 100 Qs, 25 per section, 2 marks each, -0.5 negative, 60 min").
2. **Bank sampling** — `paper_builder.py` pulls a randomized, non-repeating set of questions per topic from `questions` matching the blueprint's counts and a target difficulty mix.
3. **Gap fill (if bank is short)** — for any section where the bank can't fill the quota, `question_generator.py` calls the local LLM (via `llm_client.py`) with a strict JSON-schema prompt (question, 4 options, correct index, explanation, topic, difficulty) and syllabus context pulled via `context_manager.py`'s RAG so questions stay on-topic.

4. **Validation pass** — every LLM-generated question is checked: exactly one correct option marked, no duplicate/near-duplicate to existing bank questions (embedding similarity check), plausible option lengths. Failing questions are regenerated or dropped, never shown to a student unvalidated.
5. **Assembly** — `papers` + `paper_questions` rows are written; the paper is now immutable and reusable across multiple students taking the same exam slot.

Part 5 — New / Edited / Removed Files

Removed: `main.py`, `ui_components.py`, `sample_ui.py`, `test.py` (Flet-specific), `gayatri.db` (untrack from git).

Edited (ported into backend): `llm_client.py` → moved to `backend/services/llm_client.py`, add a `generate_structured()` method (JSON-mode + retry-on-invalid-JSON). `context_manager.py` → moved to `backend/services/context_manager.py`, repointed at `/data/syllabus/` instead of chat context files. `config.py` → split into `backend/config.py` (server settings) + `backend/config/exam_defaults.py`.

New — backend (FastAPI):

```
backend/
    main.py                          # FastAPI app entry
    db/database.py                   # SQLAlchemy engine/session
    models/                          # users, exams, questions, attempts, results (SQLAlc
  hemy)
    schemas/                         # Pydantic request/response models
    api/routes/
      auth.py  exams.py  papers.py  attempts.py  results.py  admin.py
    services/
      llm_client.py                  # ported/extended
      context_manager.py             # ported/repointed
      question_generator.py          # NEW — LLM question generation + validation
      paper_builder.py               # NEW — blueprint-driven sampling + gap-fill
      scoring.py                     # NEW — scoring, negative marking, percentile
      session_manager.py             # NEW — server-side timer/auto-submit enforcement
    ingestion/
      scrape_previous_papers.py      # NEW — RPA: source real past papers
      pdf_to_questions.py            # NEW — RPA: parse PDF papers into structured Qs
      bulk_import.py                 # NEW — RPA: CSV/JSON bank importer + schema validat
  ion
      notify.py                      # NEW — result email/notification dispatch
    tests/
```

New — frontend (React):

```
frontend/src/
    pages/
      ExamSelect.jsx  ExamInstructions.jsx  ExamTaking.jsx  ExamResult.jsx
      StudentDashboard.jsx
      AdminQuestionBank.jsx  AdminReports.jsx
    components/
      Timer.jsx  QuestionPalette.jsx  QuestionCard.jsx  ResultChart.jsx
    api/client.js
```

New — data/ops:

```
data/question_bank/<exam>/*.csv        # seed banks per exam category
  data/syllabus/<exam>/*.md              # RAG source material for gap-fill generation
  scripts/seed_db.py
  docker-compose.yml
  .env.example
  docs/ARCHITECTURE.md  API_SPEC.md  DB_SCHEMA.md  DEPLOYMENT.md
```

Part 6 — Scope of Work by Role

AI/ML Developers (2)

- Own `question_generator.py`: prompt design for structured MCQ generation, JSON-schema enforcement, retry/repair logic on malformed model output.
- Duplicate/near-duplicate detection (embedding similarity) between generated and bank questions.
- Difficulty calibration heuristics (length/vocabulary/distractor-plausibility as a first-pass proxy, refined with Data Analysts using real attempt data from Day 11+).
- Wire `context_manager.py` to feed syllabus docs into generation prompts for topical accuracy.
- Auto-explanation generation for each question (shown in results review).

Data Analysts (2)

- Own `exam_blueprints` content — research and encode real section structure, marking scheme, negative marking, and time limits for each target competitive exam.
- Design `scoring.py`'s logic: raw score, percentile calculation across attempts, topic-wise strength/weakness breakdown.
- QA the question bank for correctness and difficulty balance before it goes live (sample-review, not exhaustive).

- Build the post-launch item-analysis report (per-question difficulty index and discrimination based on real attempt data) and the admin analytics views (`AdminReports.jsx` data layer, paired with full-stack for rendering).

Full-Stack Developers (2)

- Own the FastAPI skeleton: `main.py`, all `api/routes/*`, auth, DB models/migrations.
- Own `session_manager.py` (server-side timer enforcement, auto-submit).
- Own the full React app: exam selection → instructions → timed exam-taking UI (question palette, navigation, autosave-per-answer) → result screen.
- Admin panel shell (question bank browse/edit, exam scheduling).
- Deployment: `docker-compose.yml`, environment config, hosting.

RPA & Python Developers (2)

- Build `ingestion/scrape_previous_papers.py` and `pdf_to_questions.py` to source and structure real past-year question papers into the bank format — this is what makes the bank credible instead of 100% synthetic.
- Build `bulk_import.py` with schema validation so Data Analysts can hand off CSVs safely.
- Build `scripts/seed_db.py` to populate the initial multi-exam question bank before Day 7.
- Build `notify.py` (result email dispatch) and a scheduler (APScheduler/cron) for nightly paper refresh and stale-session cleanup.

Part 7 — 15-Day Sprint Plan

Days 1–2 — Architecture lock & setup (all hands)

- Day 1: Kickoff. Walk the whole team through Part 1–5 of this doc. Finalize exam categories in scope for MVP (recommend starting with 2 exam types, not all). Repo restructure: create `backend/`, `frontend/`, `data/`, `ingestion/`; remove dead files; untrack `gayatri.db`.
- Day 2: Finalize DB schema (Full-Stack + Data Analysts) and API contract (all). Data Analysts deliver first draft `exam_blueprints` for the 2 launch exams. AI/ML sets up LM Studio structured-output prompt scaffold. RPA identifies 2–3 real past-paper sources per exam to scrape.
- **Milestone:** schema + API contract signed off, dev environments running for all 8.

Days 3–6 — Core build, Phase 1

- Full-Stack: FastAPI skeleton, DB models/migrations, auth endpoints, `exams` / `papers` read endpoints. React app skeleton + routing + `ExamSelect` / `ExamInstructions` pages against mock data.
- AI/ML: `question_generator.py` v1 (structured generation + JSON validation) working standalone against a test exam/topic.

- Data Analysts: finish blueprints for both launch exams; begin manual review criteria doc for question bank QA.
- RPA: `pdf_to_questions.py` + `scrape_previous_papers.py` producing a first structured batch; `bulk_import.py` v1.
- **Milestone (Day 6):** a question can go from "scraped/generated" → validated row in `questions` table, end to end, for at least one exam.

Days 7–10 — Core build, Phase 2

- Full-Stack: `paper_builder.py` integration (blueprint → sampled paper), `attempts` API (start/answer/submit), `session_manager.py` server-side timer + auto-submit. `ExamTaking.jsx` fully wired: timer, question palette, per-answer autosave.
- AI/ML: gap-fill generation wired into `paper_builder.py` for under-filled sections; duplicate-detection pass live.
- Data Analysts: `scoring.py` logic spec finalized and handed to Full-Stack for implementation; begin QA-reviewing the growing question bank.
- RPA: scale ingestion to full target question counts for both launch exams; seed DB via `seed_db.py`.
- **Milestone (Day 10):** a student can select an exam, get a generated paper, take it under a real timer, and submit.

Days 11–13 — Results, analytics, hardening

- Full-Stack: `results` API + `ExamResult.jsx`; admin panel (`AdminQuestionBank.jsx`) for browsing/editing bank content.
- Data Analysts: item-analysis report from Day 10's test attempts; `AdminReports.jsx` data layer (topic-wise breakdowns, percentile).
- AI/ML: auto-explanation generation for results review; difficulty calibration pass using early attempt data.
- RPA: `notify.py` (result email), scheduler for nightly refresh/cleanup.
- Cross-team: first full internal QA pass — everyone takes a mock exam end-to-end and files bugs.
- **Milestone (Day 13):** results are scored, published, and reviewable with topic-wise breakdown; internal bug list triaged.

Days 14–15 — Integration testing, deployment, handoff

- Day 14: Fix Day 13's bug list. Load-test concurrent attempts (this is where the SQLite single-writer limit gets stress-tested — document findings and a Postgres-migration recommendation rather than attempting the swap in the MVP window). Full-Stack finalizes `docker-compose.yml`/deployment.
- Day 15: Final UAT run with a real cohort of test users across both launch exams. Freeze scope, write `DEPLOYMENT.md` + `API_SPEC.md`, demo/handoff to stakeholders.
- **Milestone:** MVP deployed, 2 exam categories live end-to-end, documentation handed off.

Part 8 — Key Risks Called Out Explicitly

- **Answer-key correctness is the product's credibility.** A wrong answer key on a competitive-exam mock test destroys trust immediately — this is why Part 4 mandates a validation pass on every LLM-generated question and a QA review step owned by Data Analysts, not just AI/ML self-certifying its own output.
- **SQLite concurrency ceiling.** Fine for the 15-day MVP and a modest pilot cohort; will need a Postgres migration before any large simultaneous exam batch (e.g., a scheduled mock test with hundreds of concurrent starters). Flagged in Day 14, not silently deferred.
- **Scope discipline on exam count.** Launching with 2 exam categories instead of "various competitive exams" from day one is a recommendation, not a requirement — but attempting broad exam coverage in 15 days will thin the question bank and QA effort across too many categories at once.